

Komunikacija

Ovaj dokument opisuje arhitekturu komunikacije u realnom vremenu unutar sistema ScoreMinds, implementirane softverske komponente i protokole korišćene za sinhronizaciju podataka između različitih delova sistema.

1. Model komunikacije

Sistem koristi Event-Driven Architecture — arhitekturu zasnovanu na događajima. Umesto tradicionalnog modela gde klijent periodično zahteva podatke (Polling), primenjen je reaktivni model gde server inicira slanje podataka klijentu u trenutku nastanka promene.

- WebSockets : Koristi se za stalnu, dvosmernu vezu između Angular frontenda i NestJS backenda.
- Message Queuing : Asinhrona komunikacija između delova sistema (npr. signal za završetak meča i pokretanje kalkulacije poena).
- Interna komunikacija baze : Za automatsko održavanje konzistentnosti podataka unutar PostgreSQL baze.

2. Tehnologije i biblioteke

Za realizaciju softverske komponente odabrane su industrijski standardizovane biblioteke:

- Socket.io (Backend & Frontend):
Omogućava dvosmernu komunikaciju sa malim kašnjenjem (low-latency), poseduje ugrađen mehanizam za "Sobe" (Rooms) koji je ključan za segmentaciju saobraćaja, i automatski "fallback" na HTTP long-polling ako WebSockets nisu dostupni.
- RabbitMQ:
Omogućava razdvajanje servisa. Servis za kalkulaciju poena ne mora da zna kako je utakmica završena, samo reaguje na poruku iz reda čekanja.

3. Arhitektura softverske komponente

3.1. AppGateway

Centralna tačka real-time komunikacije je AppGateway klasa u NestJS-u. Ona implementira Pub/Sub (Publish-Subscribe) obrazac koristeći koncept soba.

3.2. CalculatingService

Ova komponenta demonstrira komunikaciju između više delova sistema (RabbitMQ -> Baza -> Socket). Logika:

1. Sluša match_finished događaj sa RabbitMQ-a.
2. Povlači predviđanja iz baze.

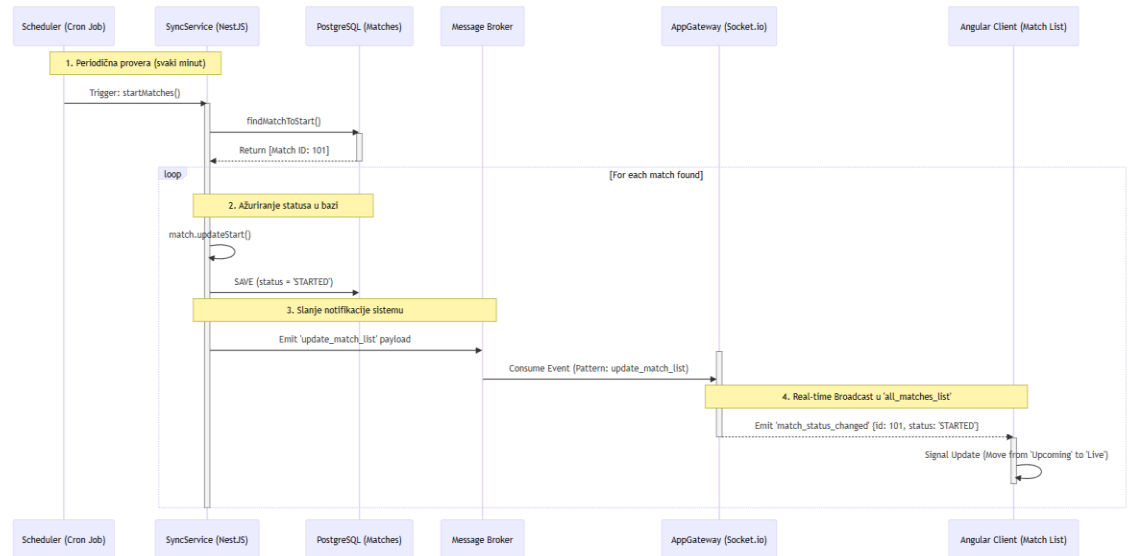
3. Računa poene na osnovu kompleksnih pravila (pobednik, tačan rezultat, događaji).
4. Šalje personalizovane notifikacije korisnicima putem Socket.io.

3.3. SyncService

Ovaj servis ima dvostruku ulogu u sistemu: prikupljanje eksternih podataka i generisanje real-time događaja za potrebe testiranja i demonstracije.

- Integracija sa eksternim API-jem: Servis komunicira sa FootballApiService radi povlačenja rasporeda utakmica i podataka o igračima. Podaci se ne upisuju direktno u bazu, već se šalju u RabbitMQ (sync_league_matches), čime se rasterećuje glavni tok aplikacije.
- Simulacija utakmica (Cron Jobs):
- @Cron(CronExpression.EVERY_MINUTE): Proverava mečeve koji treba da počnu i menja im status u STARTED.
- @Cron('* / 30 * * * *'): Simulira tok utakmice svake 30. sekunde.
- Generisanje događaja: Unutar simulacije, servis nasumično (ili po predefinisanoj logici) generiše golove i asistencije. Svaka promena stanja se odmah šalje u sistem pozivom metode notifySystem(match), koja emituje tri tipa poruka u RabbitMQ:
- update_match: Za ažuriranje detalja meča u bazi i slanje socket notifikacija u sobu utakmice.
- update_match_list: Za ažuriranje statusa na listi svih mečeva (Globalna soba).
- match_finished: Specijalna poruka koja, kada se meč završi, okida CalculatingService za obračun poena.

4. Dijagrama toka podataka za pocetak meca



Tok izvršavanja :

1. Inicijalizacija (Cron Job): Proces započinje Scheduler komponenta (NestJS Cron) koja se izvršava periodično (svakog minuta). Ona poziva metodu startMatches() u SyncService-u kako bi proverila da li postoje utakmice čije je vreme početka nastupilo.
2. Provera i Ažuriranje Baze (Data Consistency): SyncService šalje upit ka PostgreSQL bazi (findMatchToStart). Ukoliko baza vrati meč (npr. ID: 101), servis prvo ažurira status tog meča u bazi na STARTED. Ovo osigurava konzistentnost podataka pre nego što se informacija distribuira dalje.
3. Distribucija Događaja (Decoupling): Umesto da SyncService direktno poziva Socket Gateway (što bi stvorilo čvrstu vezu između modula), on šalje poruku update_match_list u Message Broker. Ovim se postiže razdvajanje (decoupling) logike simulacije od logike notifikacije.
4. Real-time Notifikacija (WebSockets):
 - a. AppGateway preuzima poruku iz brokera.
 - b. Identifikuje ciljnu grupu korisnika – u ovom slučaju sobu all_matches_list (svi korisnici koji gledaju listu utakmica).
 - c. Emituje događaj match_status_changed putem Socket.io protokola.
5. Ažuriranje Klijenta (Angular): Angular klijent prima događaj i, koristeći Signale, automatski prebacuje utakmicu iz taba "Upcoming" u tab "Live" bez osvežavanja stranice.

5. Klijenstka strana

Klijentska aplikacija koristi Angular Signals za reaktivno ažuriranje interfejsa.

- SocketService: Singleton servis koji održava konekciju

6. Obrasci

Primenjeni su State – Behavioral Pattern, Strategy (Strategija) – Behavioral Pattern i Adapter – Structural Pattern.

6.1. State – Behavioral Pattern

Opis problema:

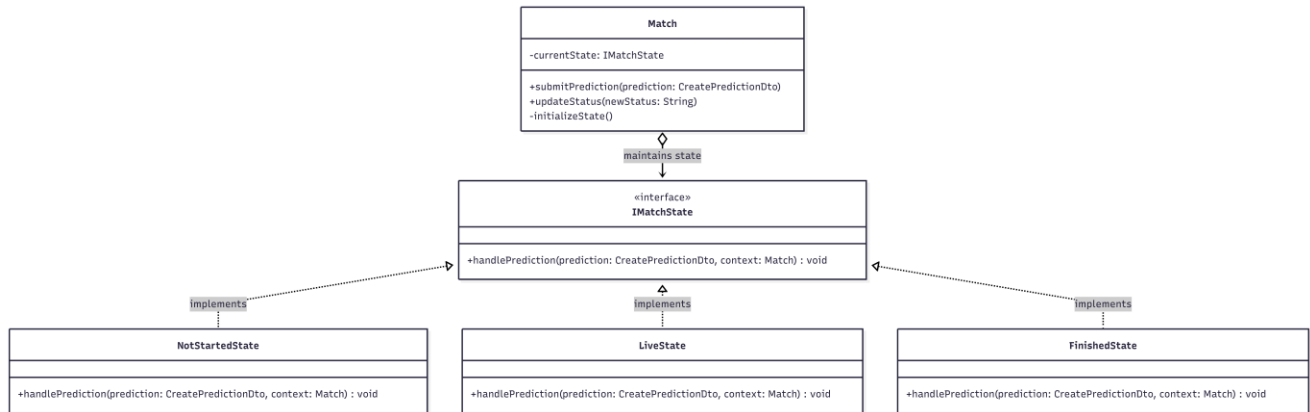
U okviru aplikacije ScoreMinds, objekat Match (Utakmica) prolazi kroz različite faze tokom svog životnog ciklusa: Not Started (Nije počela), Live (U toku) i Finished (Završena). Logika za obradu korisničkih predikcija direktno zavisi od trenutnog stanja utakmice. Na primer, postavljanje predikcije je dozvoljeno samo ako meč nije počeo. Tradicionalni pristup korišćenjem switch-case ili if-else struktura unutar servisa doveo bi do nepreglednog koda koji je težak za održavanje i testiranje.

Obrazloženje primene:

State obrazac je primenjen kako bi se ponašanje vezano za stanja enkapsuliralo u posebne klase. Time je izbegnuto kršenje Single Responsibility principa (klasa Match ne mora da brine o logici validacije) i Open-Closed principa (dodavanje novog stanja, npr. Postponed, ne zahteva promenu postojeće logike meča).

Implementacija u projektu:

- IMatchState: Interfejs koji definiše zajedničku metodu handlePrediction.
- NotStartedState: Implementira logiku koja dozvoljava unos predikcije.
- LiveState & FinishedState: Implementiraju logiku koja baca ForbiddenException ili BadRequestException, sprečavajući nelegalne akcije.
- Match: Kontekstualni objekat koji delegira pozive svom trenutnom stanju.



6.2. Strategy – Behavioral Pattern

Opis problema:

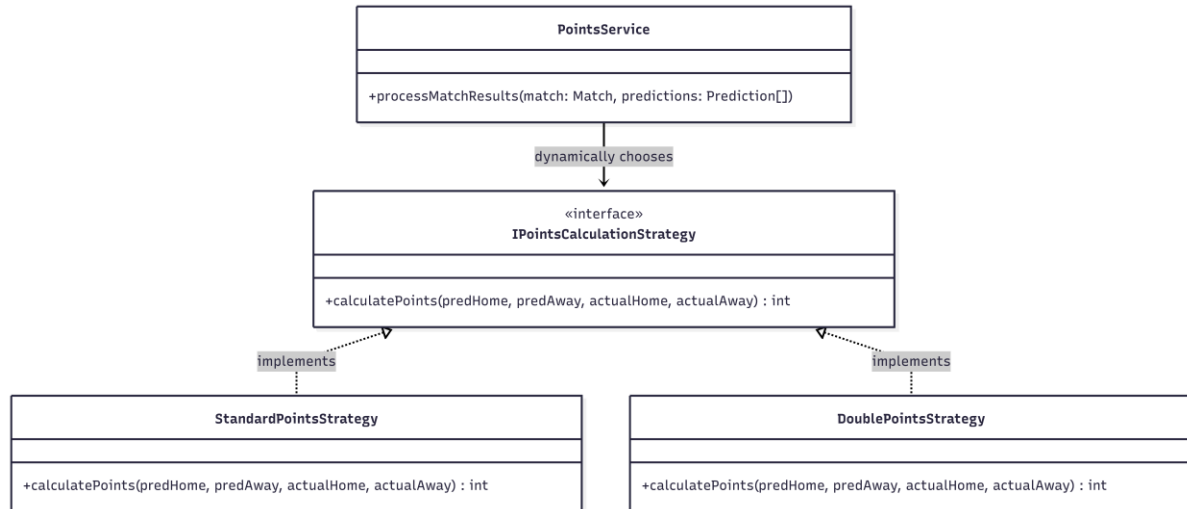
Aplikacija zahteva fleksibilan sistem za obračun poena koje korisnici osvajaju na osnovu svojih predikcija. Pravila bodovanja mogu varirati: standardna pravila za regularne mečeve ili neka dodatna pravila u zavisnosti od potrebe određenih takmicenja. Implementacija svih ovih varijacija unutar jedne funkcije bi stvorila kod koji je podložan greškama pri svakoj promeni pravila.

Obrazloženje primene:

Primenom Strategy obrasca, algoritmi za izračunavanje poena su izolovani u posebne klase. Ovo omogućava aplikaciji da u toku izvršavanja dinamički odabere odgovarajuću strategiju bodovanja u zavisnosti od konteksta meča (npr. da li je u pitanju finale). Ovime se postiže visoka modularnost i olakšava uvođenje novih sistema bodovanja bez uticaja na ostatak servisa.

Implementacija u projektu:

- **IPointsCalculationStrategy**: Interfejs sa metodom `calculatePoints`.
- **StandardPointsStrategy**: Algoritam koji dodeljuje 3 poena za tačan rezultat i 1 za ishod.
- **DoublePointsStrategy**: Algoritam namenjen finalnim utakmicama koji duplira poene.
- **PointsService**: Klijentski kod koji na osnovu atributa meča dodeljuje konkretnu strategiju objektu za obradu.



6.3. Adapter – Structural Pattern

Opis problema:

ScoreMinds preuzima podatke o utakmicama i igračima sa eksternog API servisa (API-Football). Format podataka koji ovaj API šalje je kompleksan i ne odgovara strukturi domenskih modela Match i Player koji se koriste unutar aplikacije. Direktno korišćenje stranog JSON formata u celoj aplikaciji bi stvorilo zavisnost od spoljnog provajder

Obrazloženje primene:

Adapter obrazac je iskorišćen kao "prevodilac" između eksternog API-ja i unutrašnjeg domenskog sloja. On služi kao Anti-Corruption Layer koji štiti ostatak sistema od promena u strukturi spoljnog API-ja. Ukoliko provajder promeni nazive polja u svom JSON odgovoru, izmena se vrši isključivo u klasi adaptera, dok ostatak aplikacije ostaje netaknut.

Implementacija u projektu:

- **IExternalApiAdapter**: Target interfejs koji definiše metode `adaptToMatchModel` i `adaptToPlayerModel`.
- **ApiFootballAdapter**: Konkretni adapter koji prihvata "sirove" (any) podatke sa API-ja i mapira ih u `Partial<Match>` objekte.

- SyncService: Servis koji kroz Dependency Injection koristi interfejs adaptera, ne znajući detalje implementacije specifičnog API provajdera.

